

RAP Cloud 課程 7：Service Definition / Service Binding / Publish

Lecture

這一課要證明的事

舊 On-Premise 課程 rap04 ([.claude/rules/sap-adt-mcp.md](#) 第 40.9 節) 踩過一個很隱蔽的坑：用 ADT REST API 手動 **POST** 建立 Service Binding，物件表面上建立成功 (**201 / Active**)，但缺少 Eclipse 精靈才會觸發的隱藏後端註冊步驟，導致 Publish 永遠失敗 (**Service Definition is not available**)，最後只能改用 Eclipse 精靈重建才解決。

這一課完全不會踩到這個坑——不是因為運氣好，而是**結構性避開**：這個 Cloud 環境的 MCP 工具「建立新物件」功能整個故障 ([cloud-rap-exploration](#) 已記錄)，所以從 rc01 開始，這門課的每個物件本來就**只能用** Eclipse ADT 精靈建立，沒有「抄捷徑用 API 手動建」這個選項存在。這一課要驗證的是下一層：**Publish 完成之後，服務是不是真的能用**——不只是啟用成功、不只是 ABAP Unit 通過，是拿瀏覽器直接打 OData 協定，看真實資料進得去、出得來。

Service Definition：expose 語法

查證官方文件 [ABENSRVD_DEFINE_SERVICE](#)：

```
[@service_annot1] [@service_annot2] ...  
DEFINE SERVICE service [PROVIDER CONTRACTS contract]  
{  
  EXPOSE cds_entity [AS alias];  
  ...  
}
```

這一課實際內容：

```
@EndUserText.label: 'ZRC07_SD Service Definition'  
define service ZRC07_SD {  
  expose ZI_RC01_TASK as Task;  
}
```

- **expose** 後面可以是 CDS View、CDS Abstract Entity、CDS Custom Entity；**as <alias>** 是選擇性的，但**強烈建議一律加**——OData 服務對外看到的 Entity Set 名稱就是這個別名，不加的話外部看到的會是完整的技術名稱 **ZI_RC01_TASK**，不利消費端閱讀。
- 官方文件特別提醒：如果曝露的 CDS View 有 Composition (像 rc06 的 Header-Item)，**應該把 Composition Tree 裡所有相關實體都一起曝露** (父子關聯才能被 OData 消費端正確導覽)，但**這不是語法檢查會擋的規則**，漏掉不會啟用失敗，只是執行期消費端會看不到子節點——這一課的 **ZI_RC01_TASK** 沒有 Composition，不受影響，但延伸到 rc06 的 Header-Item 結構時要記得。
- 一個 Service Definition 本身**跟通訊協定無關**，同一份 Service Definition 理論上可以被多個 Service Binding (不同協定版本) 重複使用。

Service Binding：綁定協定 + Publish

查證官方文件 [ABENCDS_SERVICE_BINDINGS](#)：Service Binding 是一個「Form-based tool」（Eclipse 專屬表單畫面）定義的 Repository 物件，把 Service Definition 綁定到一個通訊協定（OData V2 / V4，各自再分 UI / Web API 兩種用途；另外還有 InA 分析協定、SQL Service）。**OData for UI**（這一課用的）跟一般 UI 技術（SAPUI5 / Fiori Elements）搭配，資料裡會帶 UI 控制元素（由 CDS 的 `@UI.*` annotation 提供，rc08 才會加）；**OData for Web API** 純資料、不含任何 UI 控制元素，適合系統對系統整合。

這一課選 **OData V4 - UI**：

```
Binding Type: OData V4 - UI
Service Definition: ZRC07_SD
```

Service Binding 物件本身沒有原始碼可寫（不是 source-based，是純 metadata 表單），完整內容都是精靈填出來的 XML：

```
<srvb:binding srvb:type="ODATA" srvb:version="V4" srvb:category="0">
  <srvb:implementation adtcore:name="ZRC07_SB"/>
</srvb:binding>
```

`srvb:published="false"` → 按 Eclipse 編輯器的 **Publish** 按鈕後變成 `srvb:published="true"`，`srvb:allowedAction` 也從 **PUBLISH** 變成 **UNPUBLISH**——這個狀態轉換完全沒有對應的 MCP 工具可以觸發，這一課從頭到尾第一次「連一行程式碼都不用寫、但完全無法自動化」的物件。

建立流程：這一課用到的 Eclipse 操作

1. 對著 **ZI_RC01_TASK**（CDS View）右鍵 → **New** → **Service Definition**，Name **ZRC07_SD**，套用 **Define Service** 樣板 → **Finish**（樣板會自動帶出 `expose ZI_RC01_TASK;`，不含別名，Claude 事後用 MCP 補上 `as Task`）
2. 對著 **ZRC07_SD** 右鍵 → **New Service Binding**，Name **ZRC07_SB**，Binding Type 選 **OData V4 - UI** → **Finish**
3. 回到 **ZRC07_SB** 編輯器，點 **Publish**

⚠ 新踩到的環境限制：MCP 寫入偶爾卡在「Session 過期」，但單純叫使用者互動不一定夠

延續 rc05 已經記過的「`All locks dropped due to expired sessions - login failed`」——這次多踩了一次、多學到一個細節：光是請使用者在 **VS Code** 展開套件節點，不保證下次寫入就會成功（這一課實測：使用者互動過一次之後重試，還是同樣的錯誤）。真正讓寫入成功的，是 Claude 這邊額外呼叫了 `open_object` 工具把物件在 VS Code 編輯器裡開啟過一次，再重試 `replace_string_in_abap_object` 才成功。兩者是否有因果關係沒有嚴格對照實驗驗證過（使用者也持續在互動），但下次遇到同樣的鎖定錯誤，除了請使用者動一下 VS Code，也可以先用 `open_object` 主動開一次檔案再重試，作為額外的排除手段。

驗證：不用 Fiori App，直接用瀏覽器打 OData 協定

Eclipse 編輯器裡 Service Version Details 區塊除了 **Preview...**（開 Fiori Elements 臨時預覽），還有一個 **Test...** 按鈕——這是這門課第一次發現的東西：它會開一個內建的 **Swagger UI**（OpenAPI 3.0 測試介面），把整個 OData V4 服務包裝成一組可以直接互動測試的 REST 端點（`GET /Task`、`POST /Task`、`GET`

/Task/{task_id}、PATCH /Task/{task_id}、DELETE /Task/{task_id}、POST /Task/{task_id}/SAP__self.markDone)，每個端點都有 **Try it out** 可以直接送真實請求、看真實回應。

這比舊 **On-Premise** 課程能做到的驗證方式好太多：rap04 ([.claude/rules/sap-adt-mcp.md](#) 第 45 節) 想從系統內部自我呼叫驗證 OData，讀取還算能動，但寫入 (**POST**) 因為 **CSRF Token** 在多節點架構下驗證失敗、完全沒有已知 **workaround**，最後只能放棄，交給使用者手動用瀏覽器測。這一課的 Swagger UI 是 ADT 官方內建的測試工具，走的是使用者自己的瀏覽器 Session，天生沒有這個問題——**Create (POST)** 第一次嘗試就直接成功。

測試 1：GET /Task —— 確認 Publish 真的成功、資料查得到

```
GET /Task
→ 200 OK
{
  "value": [
    { "task_id": "RC02TEST03", "description": "Action test task", "status": "D", ... }
  ]
}
```

RC02TEST03 是 rc04 的 ABAP Unit 測試留下的資料——證實 OData 服務讀到的是真實資料庫內容，不是模擬資料。

⚠ **Preview** 分頁如果是 **Publish** 完成前開的，會快取住舊的 **\$metadata**，重新按 **Go** 也看不到資料——這一課第一次打開 Preview 時發生過這個狀況（畫面顯示零筆、跟 GET /Task 查到的結果矛盾），關掉分頁、重新點一次 **Preview...** 開全新分頁後才正常顯示。**判斷原則**：Publish 完成前後，任何已經開啟的 Fiori Elements Preview / Test 分頁都要重開，不能只按重新整理。

測試 2：POST /Task —— 證明 Determination 透過真正的 OData Create 也會觸發

```
POST /Task
{
  "task_id": "RC07TEST01",
  "description": "OData create test",
  "status": "O"
}
→ 201 Created
{
  "task_id": "RC07TEST01",
  "description": "OData create test",
  "status": "O",
  "created_at": "2026-08-18T13:39:40.173191Z",
  "created_by": "CB9980001705"
}
```

Request 完全沒有帶 **created_at / created_by**，回應卻自動補上了——rc03 教的 **determination setCreationInfo on save { create; }** 不是只有 EML / ABAP Unit 呼叫得到，**RAP** 框架不管是誰觸發

CUD (自訂程式的 EML、還是 OData 消費端的 HTTP 請求)，走的都是同一套 Behavior 邏輯，這是這一課第一次拿「協定層」的證據印證這件事。

測試 3：POST /Task/{task_id}/SAP__self.markDone —— 意外發現 OData V4 的 ETag / 併發控制

```
POST /Task/RC07TEST01/SAP__self.markDone
→ 428 Precondition Required
{
  "error": {
    "code": "/IWCOR/CX_OD_PRECOND_REQUIRED/...",
    "message": "The Data Service Request is required to be conditional. Try using the
    \"If-Match\" header"
  }
}
```

這不是失敗，是 OData V4 協定的標準行為，而且是一個新發現：ZI_RC01_TASK 的 BDEF 有 etag master created_at (rc03 教的樂觀鎖)——OData V4 規格規定，任何會修改一個「有 ETag」實體的請求 (PATCH / DELETE / Bound Action)，都必須帶 If-Match header，證明呼叫端是基於最新版本操作，否則一律 428 擋下。這個限制發生在 HTTP / Gateway 協定層，比 RAP Behavior Framework 更外層——這解釋了為什麼 rc02 ~ rc06 用 EML / ABAP Unit 測試從來沒遇過這個問題：EML 呼叫完全繞過 HTTP，不受 OData 協定的併發控制規則約束，只有走真正的 OData 才會摸到這一層。

這次沒有讓 Action 呼叫成功：Eclipse 內建 Swagger UI 產生的 Parameters 表格只列出 task_id (Path 參數)，沒有提供任何欄位可以加 If-Match Header——這是這個測試工具本身的限制 (Bound Action 的 OpenAPI 定義沒有把 If-Match 列成正式參數)，不是 OData 服務的問題。如果要支援自訂 Header 的工具 (Postman、curl) 測試成功，正確步驟是：① 先 GET /Task('RC07TEST01') 拿到回應裡的 @odata.etag 值 ② 呼叫 Action 時帶 If-Match: <剛才拿到的 etag 值> (或用 If-Match: * 表示不管版本強制執行)。

補充發現：BTP ABAP Environment 個人帳號不能走 Basic Auth，只能 OAuth 2.0

嘗試用 Postman 直接打真實對外網址 (瀏覽器網址列看得到的主機名稱，不是 VS Code 擴充套件本機開的 localhost:49830 Proxy) 驗證上面的 If-Match 流程時，用個人帳號 (登入 Eclipse / 瀏覽器那組) 設 Basic Auth，GET 回來的不是資料，是一段會把瀏覽器導向

...authentication.ap21.hana.ondemand.com/oauth/authorize 的 HTML / JavaScript——這是預期中的正確行為，不是設定錯誤：BTP ABAP Environment 對「個人開發者帳號」的安全設計是一律只能走 OAuth 2.0 瀏覽器登入，能用 Basic Auth 直接打的是另一種專門給系統整合用的技術帳號 (Communication User)，要另外透過 Communication Arrangement 設定才有。這跟舊 On-Premise 系統 (傳統 SAP Gateway/ICM，個人帳號本來就支援 Basic Auth) 是完全不同的認證架構，兩者不能類推。這一課沒有進一步設定 Communication Arrangement 去完成 Postman / If-Match 驗證——已經用 Swagger UI 拿到足夠扎實的證據，這條路留給有興趣延伸 BTP 通訊設定主題時再處理。

學習目標

- 能寫出 define service 語法：expose <cds_entity> as <alias>;，知道別名是選擇性但建議一律加、Composition Tree 建議一起曝露 (非語法強制)
- 知道 Service Binding 是純 metadata 的 Repository 物件 (沒有原始碼可寫)，Binding Type 有 OData V2/V4 × UI/Web API 四種主要組合，UI 版本會帶 @UI.* annotation 定義的畫面控制元素、Web API 版

本純資料

- 知道這個環境「建立新物件」的 MCP 限制，結構性避開了舊課程 rap04 踩過的「手動 API 建立 Service Binding 缺後端註冊步驟」的坑——因為這裡從一開始就沒有「用 API 抄捷徑」這個選項
- 知道 Eclipse 編輯器的 **Test...** 按鈕會開內建 Swagger UI，可以直接對 OData V4 服務送真實 GET/POST/PATCH/DELETE/Action 請求，不需要建立 Fiori App
- 能講出這一課的核心發現：Determination 透過真正的 OData Create 也會自動觸發（不只 EML）；OData V4 對有 **etag master** 的實體，修改性操作（PATCH/DELETE/Action）一律要求 **If-Match** Header，這是 HTTP 協定層的規則、不是 RAP Behavior Framework 的規則，EML 測試永遠不會碰到
- 知道 Preview / Test 分頁如果是 Publish 完成前開的，會快取舊 **\$metadata**，Publish 之後要重開新分頁才看得到正確結果
- 知道 BTP ABAP Environment 的個人開發者帳號只能走 OAuth 2.0（瀏覽器登入），不能用 Basic Auth 直接打 API；Basic Auth 是給另一種技術帳號（Communication User，透過 Communication Arrangement 設定）用的，跟 On-Premise 系統「個人帳號本來就支援 Basic Auth」是不同的架構，不能類推

物件清單

物件	名稱	型別
Service Definition	ZRC07_SD	SRVD/SRV
Service Binding (OData V4 - UI，已 Publish)	ZRC07_SB	SRVB/SVB

沿用 rc02 ~ rc04 已經建立的 **ZI_RC01_TASK**（CDS View + Managed BDEF，含 CUD / Determination / Validation / Action），這一課沒有新建 Table / CDS View / BDEF。套件：**ZRAPCLOUD**。Service Definition / Service Binding 空殼都由使用者在 Eclipse ADT 建立，Claude 用 MCP 補上 Service Definition 內容並啟用；Publish 由使用者在 Eclipse 手動完成。

驗證方式

1. **get_abap_diagnostics** 確認 **ZRC07_SD** 無語法錯誤，**abap_activate** 回報 **Activation successful**
2. **get_object_by_uri** 讀回 Service Binding metadata，確認 **srvb:published="true"**
3. Eclipse 內建 Swagger UI（Test... 按鈕）：
 - **GET /Task** → **200**，查到真實資料（**RC02TEST03**）
 - **POST /Task**（只給 **task_id/description/status**）→ **201**，**created_at/created_by** 由 Determination 自動補上
 - **POST /Task/{task_id}/SAP__self.markDone** → **428 Precondition Required**（OData V4 ETag 規則，非失敗，已用官方錯誤訊息內容確認原因）
4. Fiori Elements Preview（Publish 完成後開的全新分頁）：List Report 顯示全部 7 個欄位（因為還沒有 **@UI.*** annotation，框架預設把所有欄位當表格欄顯示），資料跟 GET /Task 查到的一致

思考題

1. 如果要用 Postman 或 **curl** 讓 **markDone** 這個 Action 真正呼叫成功，需要哪兩個步驟？（提示：先用 **GET** 拿什麼值，再用什麼 Header 帶著這個值送 Action 請求）
2. 為什麼 rc02 ~ rc06 的 ABAP Unit / EML 測試從來沒遇過這一課的 **428 Precondition Required**？這個限制是哪一層（RAP Behavior Framework / OData Gateway / HTTP 協定）在管的？
3. 如果 rc08 幫 **ZI_RC01_TASK** 加上 **@UI.*** annotation 跟 Metadata Extension，這一課看到的 Fiori Elements Preview（全部欄位當表格欄、沒有任何排版）會有什麼變化？（提示：想想 List Report 的欄

位順序 / 標籤、Object Page 的 Facet 分區)

答案

見 `zrc07_sd.srvd.abap` (Service Definition 完整內容)、`zrc07_sb.srvb.xml` (Service Binding metadata 精簡快照，含 `srvb:published="true"` 已 Publish 狀態)。SAP 端物件套件 `ZRAPCLOUD`。驗證紀錄：
Swagger UI `GET /Task 200` (查到 `RC02TEST03`)、`POST /Task 201` (`created_at/created_by` 由 Determination 自動填入)、`POST .../markDone 428` (OData V4 ETag / `If-Match` 規則，講義內文已完整解釋原因與正確呼叫方式)。